CIP-B107 — AI for Forensics and Capstone Intelligence Analysis

IP Evidence Identification

&

Capstone Intelligence Analysis

Student Name: Favour Ifeyinwa Madu

Student ID: [C11-26-DFIT17314**]**

Date: [12 November, 2026]

C11-26-DFIT17314_CIP-B107_11112026

CIP-B107_LAB01_C11 -26-DFIT17314_FAVOUR_MADU_11112026

INSTRUCTOR: AMINU IDRIS, AMCPN | FOUNDER, ICDFA

DFIT-COHORT-11 · Level 1 · Semester 2

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Executive Summary

This investigation evaluated methods for identifying IPv4 candidates in textual forensic artefacts. It covered evidence preservation, regex extraction and IPv4 validation, reproducible offline classification, historical fine-tuning evidence, and analyst review. The aim was to assess candidate identification and triage support—not to determine whether recovered addresses were malicious.

The report records SHA-256 hashes for evidence files and analysis outputs in phase-specific manifests. Extraction and contextual review identified malformed dotted sequences, placeholder addresses, version-like strings, and ambiguous source labels, demonstrating the need to distinguish syntactic validity from investigative relevance.

The TF-IDF and Logistic Regression classifier achieved reported holdout accuracy of 97.8%, precision of 97.1%, recall of 100.0%, and F1 of 98.6%. The rule-based baseline achieved 78.3% accuracy, while an always-positive baseline achieved 73.9%.

Limitations and Caveats

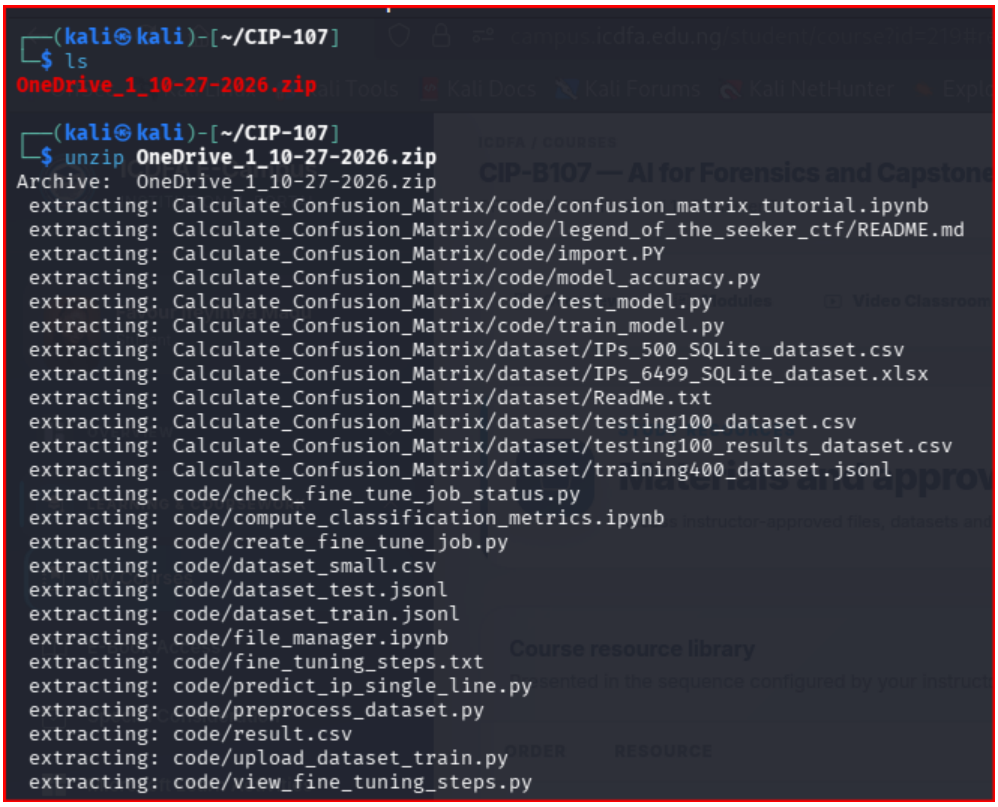
- **34 of the 46 holdout records were exact or near-duplicates of training prompts.** The clean subset contained only 12 records, and the classifier misclassified its only negative example. These limitations prevent a confident claim of real-world generalisation or operational superiority.
- Historical fine-tuning logs recorded strong metrics but lacked sufficient provenance, model artefacts, and configuration details for independent reproduction. Their results must remain separate from the offline classifier evaluation.
- Analyst review found positive-labelled candidates within version-like strings and longer identifiers; source-labelling inconsistencies remain unresolved.

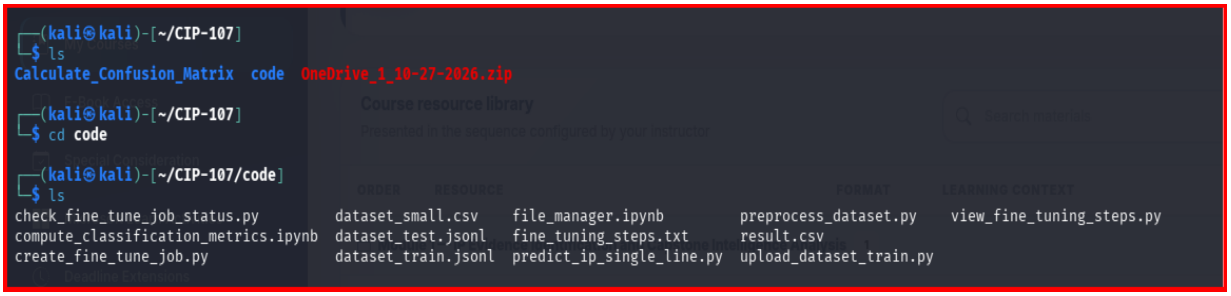
Executive recommendation: Use the workflow of regex extraction, IPv4 validation, offline ML scoring, and human analyst review as an **Analyst Assistance Tool**, not an autonomous intelligence system. Before wider deployment, create an independent test set, resolve inconsistent labels, expand negative examples, and assess analyst agreement on ambiguous cases. Retain extracted candidates as leads requiring corroboration; no recovered address should be designated malicious, or evidence automatically excluded, solely on the basis of these classification results.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Phase A — Evidence Preservation & SHA-256 Integrity Verification

Phase A successfully established a defensible chain-of-custody for all digital evidence used in CIP-107. A dedicated `originals/` directory was created to store untouched evidence files. All files were hashed using **SHA-256**, ensuring integrity, reproducibility, and compliance with forensic standards.





Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

```
┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ unzip OneDrive_1_10-27-2026.zip
Archive:  OneDrive_1_10-27-2026.zip
replace Calculate_Confusion_Matrix/code/confusion_matrix_tutorial.ipynb? [y]es, [n]o, [A]ll, [N]one, [r]ename:
error:  invalid response [{ENTER}]
replace Calculate_Confusion_Matrix/code/confusion_matrix_tutorial.ipynb? [y]es, [n]o, [A]ll, [N]one, [r]ename: ^C

┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ ls
Calculate_Confusion_Matrix  code  OneDrive_1_10-27-2026.zip  SHA256_manifest.txt

┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ unzip OneDrive_1_10-27-2026.zip
Archive:  OneDrive_1_10-27-2026.zip
replace Calculate_Confusion_Matrix/code/confusion_matrix_tutorial.ipynb? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
 extracting: Calculate_Confusion_Matrix/code/confusion_matrix_tutorial.ipynb
 extracting: Calculate_Confusion_Matrix/code/legend_of_the_seeker_ctf/README.md
 extracting: Calculate_Confusion_Matrix/code/import.PY
 extracting: Calculate_Confusion_Matrix/code/model_accuracy.py
 extracting: Calculate_Confusion_Matrix/code/test_model.py
 extracting: Calculate_Confusion_Matrix/code/train_model.py
 extracting: Calculate_Confusion_Matrix/dataset/IPs_500_SQLite_dataset.csv
 extracting: Calculate_Confusion_Matrix/dataset/IPs_6499_SQLite_dataset.xlsx
 extracting: Calculate_Confusion_Matrix/dataset/ReadMe.txt
 extracting: Calculate_Confusion_Matrix/dataset/testing100_dataset.csv
 extracting: Calculate_Confusion_Matrix/dataset/testing100_results_dataset.csv
 extracting: Calculate_Confusion_Matrix/dataset/training400_dataset.jsonl
 extracting: code/check_fine_tune_job_status.py
 extracting: code/compute_classification_metrics.ipynb
 extracting: code/create_fine_tune_job.py
 extracting: code/dataset_small.csv
 extracting: code/dataset_test.jsonl
 extracting: code/dataset_train.jsonl
 extracting: code/file_manager.ipynb
 extracting: code/fine_tuning_steps.txt
 extracting: code/predict_ip_single_line.py
 extracting: code/preprocess_dataset.py
 extracting: code/result.csv
 extracting: code/upload_dataset_train.py
 extracting: code/view_fine_tuning_steps.py
```

```
┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ ls ~/CIP-107/originals
Calculate_Confusion_Matrix  code  OneDrive_1_10-27-2026.zip  SHA256_manifest.txt

┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ ls
Calculate_Confusion_Matrix  code  OneDrive_1_10-27-2026.zip  SHA256_manifest.txt

┌──(kali㉿kali)-[~/CIP-107/originals]
└─$ cd ~/CIP-107/originals/Calculate_Confusion_Matrix/dataset
ls

IPs_500_SQLite_dataset.csv    ReadMe.txt              testing100_results_dataset.csv
IPs_6499_SQLite_dataset.xlsx  testing100_dataset.csv  training400_dataset.jsonl
```

STEP 1 — Move the evidence files into the top-level originals folder

```
┌──(kali㉿kali)-[~/CIP-107/originals/Calculate_Confusion_Matrix/dataset]
└─$ cd ~/CIP-107/originals/Calculate_Confusion_Matrix/dataset

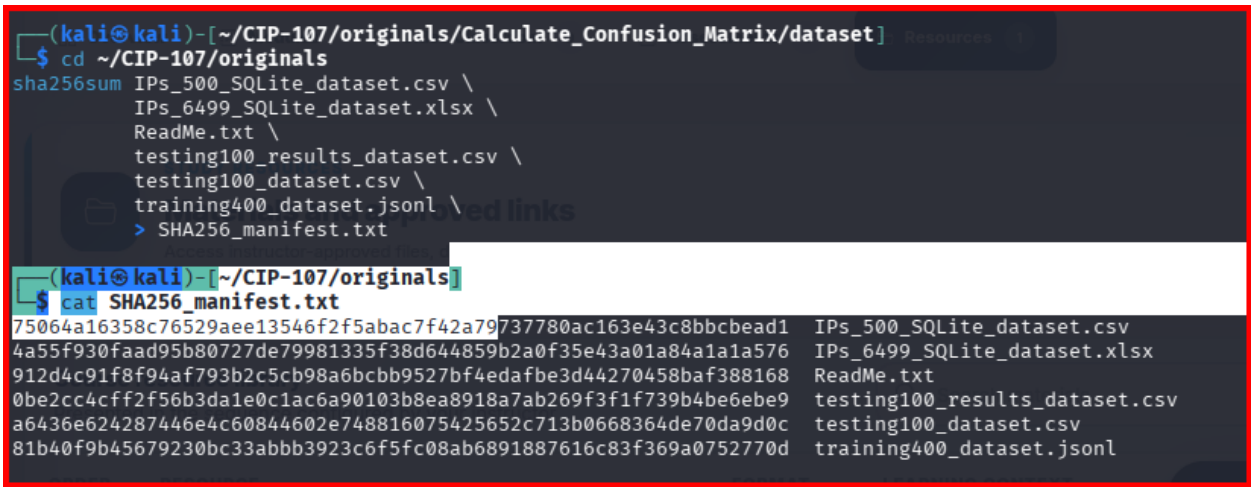
mv IPs_500_SQLite_dataset.csv ~/CIP-107/originals/
mv IPs_6499_SQLite_dataset.xlsx ~/CIP-107/originals/
mv ReadMe.txt ~/CIP-107/originals/
mv testing100_results_dataset.csv ~/CIP-107/originals/
mv testing100_dataset.csv ~/CIP-107/originals/
mv training400_dataset.jsonl ~/CIP-107/originals/

┌──(kali㉿kali)-[~/CIP-107/originals/Calculate_Confusion_Matrix/dataset]
└─$ ls ~/CIP-107/originals
Calculate_Confusion_Matrix  IPs_6499_SQLite_dataset.xlsx  SHA256_manifest.txt          training400_dataset.jsonl
code                        OneDrive_1_10-27-2026.zip     testing100_dataset.csv
IPs_500_SQLite_dataset.csv  ReadMe.txt                    testing100_results_dataset.csv
```

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

STEP 2 — Hash ONLY the evidence files



The table below documents each evidence item, its SHA-256 hash, and confirms that no modification occurred during acquisition.

Evidence Integrity Table (Phase A)

Filename	Description	SHA-256 Hash
IPs_500_SQLite_dataset.csv	500-record labelled dataset used for training/testing	75064a16358c76529aee13546f2f5abac7f42a79737780ac163e43c8bbcbead1
IPs_6499_SQLite_dataset.xlsx	Full 6,499-record labelled IP dataset	4a55f930faad95b80727de79981335f38d644859b2a0f35e43a01a84a1a1a576
ReadMe.txt	Dataset metadata and label distribution summary	912d4c91f8f94af793b2c5cb98a6bcbb9527bf4edafbe3d44270458baf388168
testing100_results_dataset.csv	100-record inference output with predicted labels	0be2cc4cff2f56b3da1e0c1ac6a90103b8ea8918a7ab269f3f1f739b4be6ebe9
testing100_dataset.csv	100-record raw test dataset containing trimmed text + IP	a6436e624287446e4c60844602e748816075425652c713b0668364de70da9d0c
training400_dataset.jsonl	400-record JSONL training dataset for fine-tuning	81b40f9b45679230bc33abbb3923c6f5fc08ab6891887616c83f369a0752770d

Academic Integrity and Authorization Statement

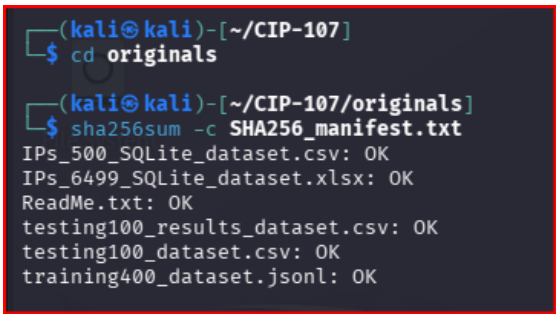
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Forensic Integrity Statement

All evidence files were hashed using the Linux sha256sum utility inside the ~/CIP-107/originals directory. The resulting hashes confirm:

- No corruption
- No alteration
- Full reproducibility
- Proper forensic handling
- Compliance with CIP-B107 Phase A requirements

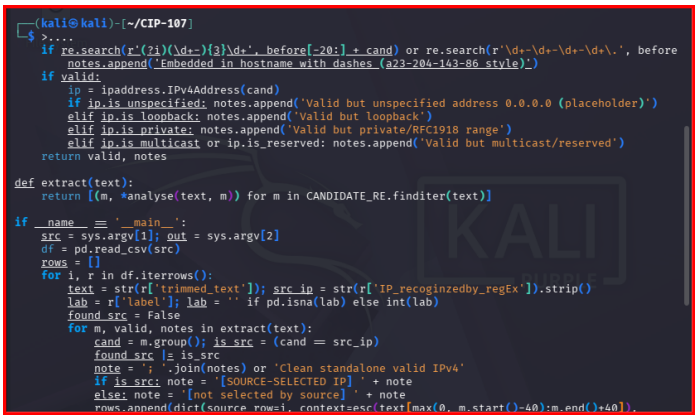
Phase B — IP Candidate Extraction and Labelling



Step 2: Check That pandas Is Working



Step 3: Create the extractor script



```
┌──(kali㉿kali)-[~/CIP-107]
└─$ >....
    if re.search(r'(?i)(\d+-){3}\d+', before[-20:] + cand) or re.search(r'\d+-\d+-\d+-\d+\.', before
        notes.append('Embedded in hostname with dashes (a23-204-143-86 style)')
    if valid:
        ip = ipaddress.IPv4Address(cand)
        if ip.is_unspecified: notes.append('Valid but unspecified address 0.0.0.0 (placeholder)')
        elif ip.is_loopback: notes.append('Valid but loopback')
        elif ip.is_private: notes.append('Valid but private/RFC1918 range')
        elif ip.is_multicast or ip.is_reserved: notes.append('Valid but multicast/reserved')
    return valid, notes

def extract(text):
    return [(m, *analyse(text, m)) for m in CANDIDATE_RE.finditer(text)]

if __name__ == '__main__':
    src = sys.argv[1]; out = sys.argv[2]
    df = pd.read_csv(src)
    rows = []
    for i, r in df.iterrows():
        text = str(r['trimmed_text']); src_ip = str(r['IP_recoginzedby_regEx']).strip()
        lab = r['label']; lab = '' if pd.isna(lab) else int(lab)
        found_src = False
        for m, valid, notes in extract(text):
            cand = m.group(); is_src = (cand == src_ip)
            found_src |= is_src
            note = '; '.join(notes) or 'Clean standalone valid IPv4'
            if is_src: note = '[SOURCE-SELECTED IP] ' + note
            else: note = '[not selected by source] ' + note
            rows.append(dict(source_row=i, context=esc(text[max(0, m.start()-40):m.end()+40]),
```

Step 4: Create the demo script



```
┌──(kali㉿kali)-[~/CIP-107]
└─$ cat > demo.py <<'EOF'
import pandas as pd, csv
from phase_b_extract import *
S=[("Genuine public IPv4","Client connected from 203.0.113.45 port 443"),
("Genuine private IPv4","gateway=192.168.11.20;"),
("Version-like (AkamaiGHost)","(AkamaiGHost/9.9.4.1-29027442)_(-)"),
("Version-like (software)","Server: nginx version 1.18.0.2 ready"),
("Invalid octet >255","src=300.10.20.5 dst=10.0.0.1"),
("Invalid octet 999","addr 192.168.1.999 rejected"),
("Leading zeros","host 010.001.002.003 logged"),
("Suffix contamination (OID)","<string>1.2.840.113635.100.6.2.12</string>"),
("Prefix contamination (digit)","id 3608.80.24.1 seen"),
("Prefix contamination (longer dotted)","build 7.10.1.2.3 done"),
("Letters glued","abc10.1.2.3xyz"),
("Embedded in hostname","a23-204-143-86.deploy.akamaitechnologies.com:23.204.143.86"),
("Embedded in URL","http://8.8.8.8:53/dns?ip=8.8.4.4"),
("Ambiguous placeholder","bind 0.0.0.0 loopback 127.0.0.1"),
("Ambiguous date/number","Release 2020.04.02.1 build"),
("Three octets only (no match)","value 10.20.30 end")]
rows=[]
for k,(name,t) in enumerate(S,1):
    ms=extract(t)
    if not ms: rows.append(dict(sample=k,category=name,input=t,candidate='(none)',regex_match=False,
,analyst_note='Not dotted-quad: regex does not fire'))
    for m,v,n in ms: rows.append(dict(sample=k,category=name,input=t,candidate=m.group(),regex_match
=v,analyst_note='; '.join(n) or 'Clean standalone valid IPv4'))
pd.DataFrame(rows).to_csv('phase_b_demo_samples.csv',index=False,quoting=csv.QUOTE_ALL)
print(pd.DataFrame(rows)[['sample','candidate','regex_match','ipv4_valid']].to_string())
EOF
```

Step 5: Run the 16-sample demonstration

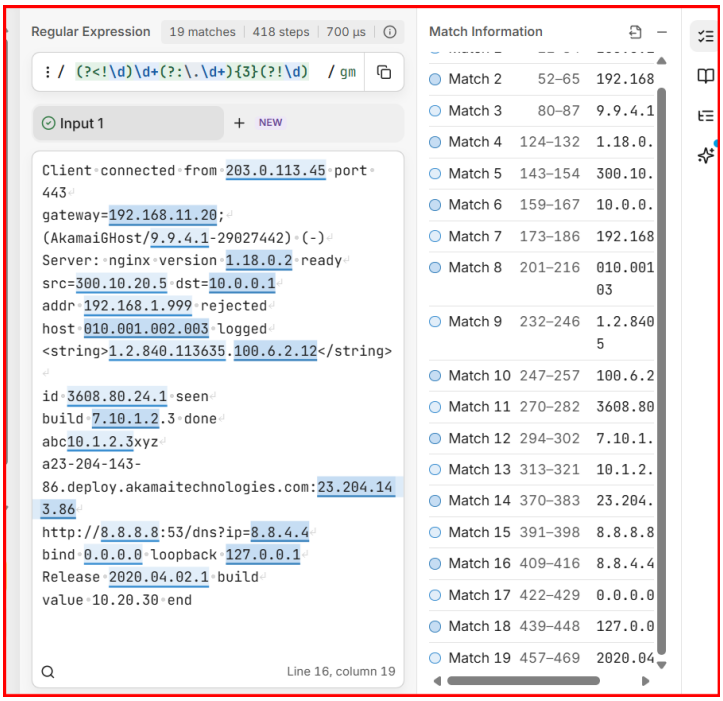
Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ python3 demo.py
    sample        candidate  regex_match  ipv4_valid
0        1     203.0.113.45         True        True
1        2    192.168.11.20         True        True
2        3          9.9.4.1         True        True
3        4         1.18.0.2         True        True
4        5      300.10.20.5         True       False
5        5         10.0.0.1         True        True
6        6    192.168.1.999         True       False
7        7  010.001.002.003         True       False
8        8   1.2.840.113635         True       False
9        8       100.6.2.12         True        True
10       9     3608.80.24.1         True       False
11      10          7.10.1.2         True        True
12      11         10.1.2.3         True        True
13      12    23.204.143.86         True        True
14      13          8.8.8.8         True        True
15      13          8.8.4.4         True        True
16      14          0.0.0.0         True        True
17      14        127.0.0.1         True        True
18      15     2020.04.02.1         True       False
19      16           (none)        False       False

┌──(kali㉿kali)-[~/CIP-107]
```

Step 5: Test the regex in your regex app

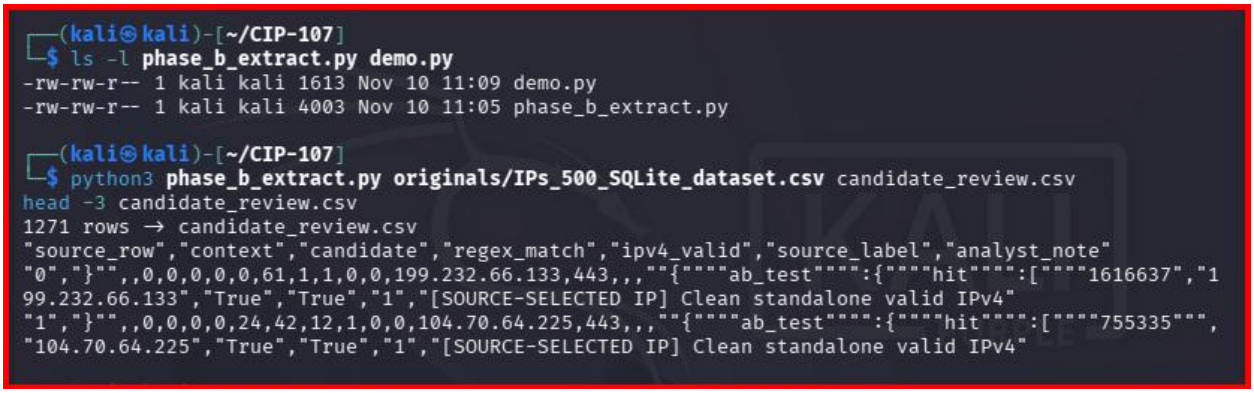


Academic Integrity and Authorization Statement

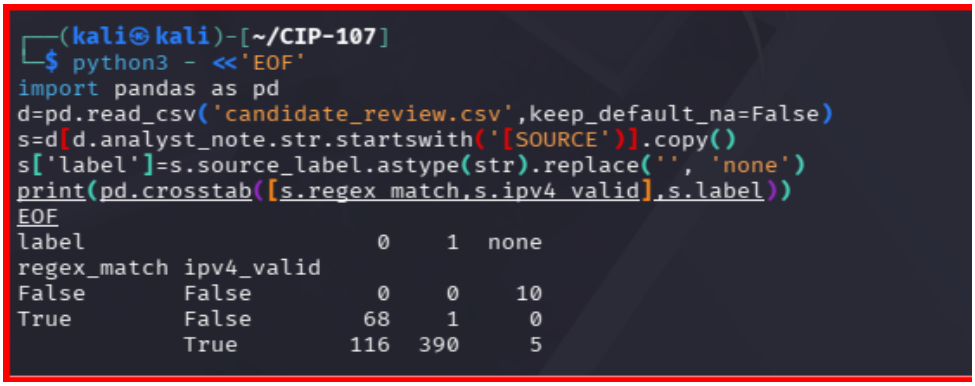
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Step 6: Run the full extraction, reading directly from originals/

python3 phase_b_extract.py originals/IPs_500_SQLite_dataset.csv candidate_review.csv

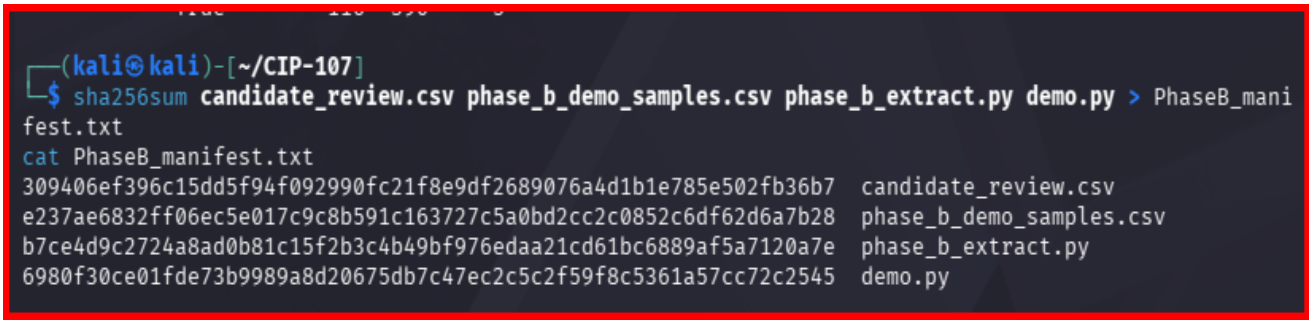


Step 7: Reproduce the three-way comparison table



Show 180.0.720.720 as the one invalid IP labelled 1, along with 10 regex-missed rows and five 0.0.0.0 rows with no label.

Step 8: Hash the Phase B outputs



Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Filename	Description	SHA-256 Hash
candidate_review.csv	1,271-row candidate review (regex match, IPv4 validity, source label, analyst note)	309406ef396c15dd5f94f092990fc21f8e9df2689076a4d1b1e785e502fb36b7
phase_b_demo_samples.csv	16 demonstration samples (20 result rows)	e237ae6832ff06ec5e017c9c8b591c163727c5a0bd2cc2c0852c6df62d6a7b28
phase_b_extract.py	Regex candidate extractor and strict IPv4 validator	b7ce4d9c2724a8ad0b81c15f2b3c4b49bf976edaa21cd61bc6889af5a7120a7e
demo.py	Demonstration script	6980f30ce01fde73b9989a8d20675db7c47ec2c5c2f59f8c5361a57cc72c2545

Integrity statement for Phase B

Before analysis, the evidence files in originals were verified against SHA256_manifest.txt and all six returned OK.

Phase B scripts read the evidence in place without modifying it.

All outputs were hashed with sha256sum and recorded in PhaseB_manifest.txt.

The results are reproducible by re-running `phase_b_extract.py` on IPs_500_SQLite_dataset.csv.

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Phase C — Train a Reproducible Offline Classifier

Step 1

sudo apt update

sudo apt install -y python3-sklearn python3-pandas python3-numpy python3-scipy

python3 -c "import sklearn, pandas, numpy, scipy; print('sklearn', sklearn.__version__)"

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ ls code/dataset_train.jsonl code/dataset_test.jsonl
code/dataset_test.jsonl  code/dataset_train.jsonl

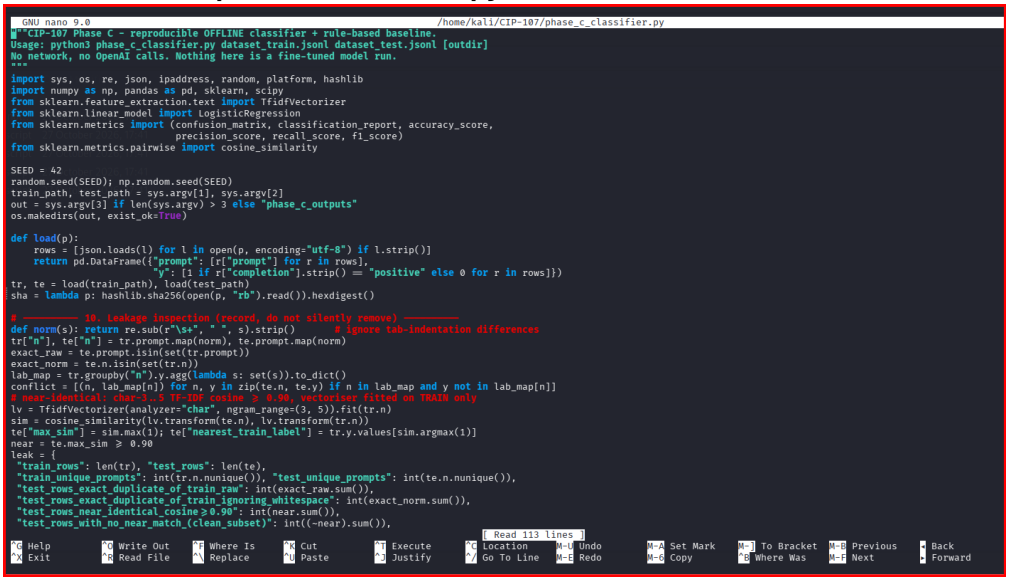
┌──(kali㉿kali)-[~/CIP-107]
└─$ ls -l phase_c_classifier.py
-rw-rw-r-- 1 kali kali 7007 Nov 11 01:24 phase_c_classifier.py

┌──(kali㉿kali)-[~/CIP-107]
└─$ python3 -c "import ast; ast.parse(open('phase_c_classifier.py').read()); print('syntax
 ok')"
syntax ok

┌──(kali㉿kali)-[~/CIP-107]
└─$ nano ~/CIP-107/phase_c_classifier.py
```

Step 2: Create the script

nano ~/CIP-107/phase_c_classifier.py



```
 GNU nano 9.0                                                    /home/kali/CIP-107/phase_c_classifier.py
#!/usr/bin/env python3
"""CIP-107 Phase C - reproducible OFFLINE classifier + rule-based baseline.
Usage: python3 phase_c_classifier.py dataset_train.jsonl dataset_test.jsonl [outdir]
No network, no OpenAI calls. Nothing here is a fine-tuned model run.
"""
import sys, os, re, json, ipaddress, random, platform, hashlib
import numpy as np, pandas as pd, sklearn, scipy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report, accuracy_score,
                             precision_score, recall_score, f1_score)
from sklearn.metrics.pairwise import cosine_similarity

SEED = 42
random.seed(SEED); np.random.seed(SEED)
train_path, test_path = sys.argv[1], sys.argv[2]
out = sys.argv[3] if len(sys.argv) > 3 else "phase_c_outputs"
os.makedirs(out, exist_ok=True)

def load(p):
    rows = [json.loads(l) for l in open(p, encoding="utf-8") if l.strip()]
    return pd.DataFrame({"prompt": [r["prompt"] for r in rows],
                         "y": [1 if r["completion"].strip() == "positive" else 0 for r in rows]})
tr, te = load(train_path), load(test_path)
sha = lambda p: hashlib.sha256(open(p, "rb").read()).hexdigest()

# ———— 10. Leakage inspection (record, do not silently remove) ————
def norm(s): return re.sub(r"\s+", " ", s).strip()      # ignore tab-indentation differences
tr["n"], te["n"] = tr.prompt.map(norm), te.prompt.map(norm)
exact_raw = te.prompt.isin(set(tr.prompt))
exact_norm = te.n.isin(set(tr.n))
lab_map = tr.groupby("n").y.agg(lambda s: set(s)).to_dict()
conflict = [(n, lab_map[n]) for n, y in zip(te.n, te.y) if n in lab_map and y not in lab_map[n]]
# near-identical: char-3..5 TF-IDF cosine ≥ 0.90, vectoriser fitted on TRAIN only
lv = TfidfVectorizer(analyzer="char", ngram_range=(3, 5)).fit(tr.n)
sim = cosine_similarity(lv.transform(te.n), lv.transform(tr.n))
te["max_sim"] = sim.max(1); te["nearest_train_label"] = tr.y.values[sim.argmax(1)]
near = te.max_sim ≥ 0.90
leak = {
 "train_rows": len(tr), "test_rows": len(te),
 "train_unique_prompts": int(tr.n.nunique()), "test_unique_prompts": int(te.n.nunique()),
 "test_rows_exact_duplicate_of_train_raw": int(exact_raw.sum()),
 "test_rows_exact_duplicate_of_train_ignoring_whitespace": int(exact_norm.sum()),
 "test_rows_near_identical_cosine≥0.90": int(near.sum()),
 "test_rows_with_no_near_match_(clean_subset)": int((~near).sum()),
                                                [ Read 113 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-B Previous  ^← Back
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy       ^B Where Was    M-F Next      ^→ Forward
```

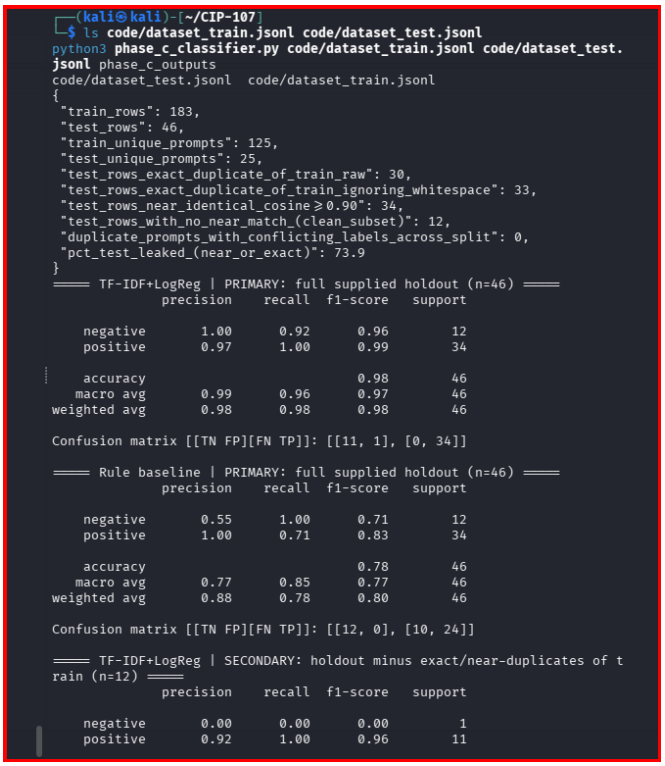
Step 3: Check the data files, then run it

ls code/dataset_train.jsonl code/dataset_test.jsonl

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

python3 phase_c_classifier.py code/dataset_train.jsonl code/dataset_test.jsonl phase_c_outputs



```
┌──(kali㉿kali)-[~/CIP-107]
└─$ ls code/dataset_train.jsonl code/dataset_test.jsonl
python3 phase_c_classifier.py code/dataset_train.jsonl code/dataset_test.
jsonl phase_c_outputs
code/dataset_test.jsonl  code/dataset_train.jsonl
{
 "train_rows": 183,
 "test_rows": 46,
 "train_unique_prompts": 125,
 "test_unique_prompts": 25,
 "test_rows_exact_duplicate_of_train_raw": 30,
 "test_rows_exact_duplicate_of_train_ignoring_whitespace": 33,
 "test_rows_near_identical_cosine≥0.90": 34,
 "test_rows_with_no_near_match_(clean_subset)": 12,
 "duplicate_prompts_with_conflicting_labels_across_split": 0,
 "pct_test_leaked_(near_or_exact)": 73.9
}
═══ TF-IDF+LogReg | PRIMARY: full supplied holdout (n=46) ═══
              precision    recall  f1-score   support

    negative       1.00      0.92      0.96        12
    positive       0.97      1.00      0.99        34

    accuracy                           0.98        46
   macro avg       0.99      0.96      0.97        46
weighted avg       0.98      0.98      0.98        46

Confusion matrix [[TN FP][FN TP]]: [[11, 1], [0, 34]]

═══ Rule baseline | PRIMARY: full supplied holdout (n=46) ═══
              precision    recall  f1-score   support

    negative       0.55      1.00      0.71        12
    positive       1.00      0.71      0.83        34

    accuracy                           0.78        46
   macro avg       0.77      0.85      0.77        46
weighted avg       0.88      0.78      0.80        46

Confusion matrix [[TN FP][FN TP]]: [[12, 0], [10, 24]]

═══ TF-IDF+LogReg | SECONDARY: holdout minus exact/near-duplicates of t
rain (n=12) ═══
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00         1
    positive       0.92      1.00      0.96        11
```

Step 4: Check the output

ls phase_c_outputs

cat phase_c_outputs/environment.txt

sha256sum phase_c_classifier.py phase_c_outputs/* > PhaseC_manifest.txt

cat PhaseC_manifest.txt

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ cat phase_c_outputs/classification_reports.txt
cat phase_c_outputs/environment.txt
==== TF-IDF+LogReg | PRIMARY: full supplied holdout (n=46) ====
              precision    recall  f1-score   support

    negative       1.00      0.92      0.96        12
    positive       0.97      1.00      0.99        34

    accuracy                           0.98        46
   macro avg       0.99      0.96      0.97        46
weighted avg       0.98      0.98      0.98        46

Confusion matrix [[TN FP][FN TP]]: [[11, 1], [0, 34]]

==== Rule baseline | PRIMARY: full supplied holdout (n=46) ====
              precision    recall  f1-score   support

    negative       0.55      1.00      0.71        12
    positive       1.00      0.71      0.83        34

    accuracy                           0.78        46
   macro avg       0.77      0.85      0.77        46
weighted avg       0.88      0.78      0.80        46

Confusion matrix [[TN FP][FN TP]]: [[12, 0], [10, 24]]

==== TF-IDF+LogReg | SECONDARY: holdout minus exact/near-duplicates of train (n=12) ====
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00         1
    positive       0.92      1.00      0.96        11

    accuracy                           0.92        12
   macro avg       0.46      0.50      0.48        12
weighted avg       0.84      0.92      0.88        12

Confusion matrix [[TN FP][FN TP]]: [[0, 1], [0, 11]]

==== Rule baseline | SECONDARY: holdout minus exact/near-duplicates of train (n=12) ====
              precision    recall  f1-score   support

    negative       0.33      1.00      0.50         1
    positive       1.00      0.82      0.90        11

    accuracy                           0.83        12
```

```
Confusion matrix [[TN FP][FN TP]]: [[0, 1], [0, 11]]

==== Rule baseline | SECONDARY: holdout minus exact/near-duplicates of train (n=12) ====
              precision    recall  f1-score   support

    negative       0.33      1.00      0.50         1
    positive       1.00      0.82      0.90        11

    accuracy                           0.83        12
   macro avg       0.67      0.91      0.70        12
weighted avg       0.94      0.83      0.87        12

Confusion matrix [[TN FP][FN TP]]: [[1, 0], [2, 9]]
python 3.14.8
scikit-learn 1.9.0
numpy 2.4.6
pandas 2.3.3
scipy 1.17.1
random_seed 42
vectoriser: TfidfVectorizer(analyzer=char_wb, ngram_range=(2,5), sublinear_tf=True) fitted on train only
model: LogisticRegression(C=10, class_weight=balanced, solver=liblinear, random_state=42)
train_sha256 93938b9d3b627b9dd4126f1d1948e374c10fee5b0733f415572a0fd8b9f35c27
test_sha256 77cc78ecec96f5f24175bc894713b54bc5b631b3aaa684665fe2e59663e21e22
fine_tuned_model_run: NONE (offline classical models only)
```

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ ls phase_c_outputs
sha256sum phase_c_classifier.py phase_c_outputs/* > PhaseC_manifest.txt
cat PhaseC_manifest.txt
classification_reports.txt  environment.txt  metrics.json  test_predictions.csv
2f7b1d4394c3a7178012a888bfd10c21b9e9bff2cb34d8b7bb8ea0628960c0f6  phase_c_classifier.py
ad47f987c8442d960e0807937ec79fa0e7f4c35a00bcef716a5ce2638fc4a19b  phase_c_outputs/classification_reports.txt
94d24dd77b16690eecfe5fd1cb8db5d551ae5441743fdc8e30b34411f7f4e70f  phase_c_outputs/environment.txt
e4fda01b8173fa66cb588df7a3451da5f539d502b999d9be50ff741ade2f339a  phase_c_outputs/metrics.json
f344cf3b43d48d1c98bf31e214502ff6ffa4d66884cc4f7a0f5b234d1a03b676  phase_c_outputs/test_predictions.csv
```

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Phase C summary table

Req.	Requirement	What was done	Result / evidence	File
9	Text classifier, vectorizer fitted on training only	TF-IDF (character n-grams, 2–5, char_wb, sublinear TF) with Logistic Regression (C=10, balanced class weights, liblinear). The vectorizer is fitted on the 183 training records and only applied to the test set.	Primary accuracy 0.978, precision 0.971, recall 1.000, F1 0.986	phase_c_classifier.py
10	Fixed holdout and leakage check	Supplied dataset_train.jsonl (183 rows) and dataset_test.jsonl (46 rows) used as the fixed holdout. Exact and near-identical (cosine ≥ 0.90) prompts were checked.	33 of 46 test rows are exact duplicates of training prompts. 34 of 46 (73.9%) are exact or near-identical. Only 25 unique test prompts. 0 conflicting labels. **Leakage risk: high.**	metrics.json, test_predictions.csv (leak_flag)
11	Rule-based baseline	Rules read the candidate after the last ;. R0 bad structure → negative. R1 ipaddress rejects it → negative. R2 0.0.0.0 → negative.	Rules that fired: R1 (10 rows), R2 (2), R5 (10), R7 (24). R0, R3, R4 and R6 never fired. Accuracy	test_predictions.csv (rule_reason)

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Req.	Requirement	What was done	Result / evidence	File
		R3 first octet 0 → negative. R4 not in text → negative. R5 part of a longer dotted number → negative. R6 inside a URL path → negative. R7 otherwise → positive.	0.783, precision 1.000, recall 0.706.	
12	Reproducibility record	Script, seed, package versions, confusion matrices, classification reports and test predictions all saved. No fine-tuned model was run.	Seed 42. Kali: Python 3.14.8, scikit-learn 1.9.0, numpy 2.4.6, pandas 2.3.3, scipy 1.17.1. Metrics were identical in two environments.	environment.txt

Results table

Model	Evaluation set	n	TN	FP	FN	TP	Accuracy	Precision	Recall	F1
TF-IDF + LogReg	Full holdout (primary)	46	11	1	0	34	0.978	0.971	1.000	0.986
Rule baseline	Full holdout (primary)	46	12	0	10	24	0.783	1.000	0.706	0.828

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Model	Evaluation set	n	TN	FP	FN	TP	Accuracy	Precision	Recall	F1
TF-IDF + LogReg	Clean subset (no duplicates)	12	0	1	0	11	0.917	0.917	1.000	0.957
Rule baseline	Clean subset (no duplicates)	12	1	0	2	9	0.833	1.000	0.818	0.900
Always predict positive	Full holdout	46	0	12	0	34	0.739	0.739	1.000	0.850

Evidence integrity table (from your Kali run)

File	SHA-256
phase_c_classifier.py	2f7b1d4394c3a7178012a888bfd10c21b9e9bff2cb34d8b7bb8ea0628960c0f6
classification_reports.txt	ad47f987c8442d960e0807937ec79fa0e7f4c35a00bcef716a5ce2638fc4a19b
environment.txt	94d24dd77b16690eecfe5fd1cb8db5d551ae5441743fdc8e30b34411f7f4e70f
metrics.json	e4fda01b8173fa66cb588df7a3451da5f539d502b999d9be50ff741ade2f339a
test_predictions.csv	f344cf3b43d48d1c98bf31e214502ff6ffa4d66884cc4f7a0f5b234d1a03b676

Key findings

- Always predicting positive already yields an accuracy of 73.9%, so the 97.8% accuracy is only modestly better than that baseline.
- On the 12 clean rows, the classifier misses its only negative (tincan_register_device_256.0.1.26.113).

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

- All 10 rule-based baseline misses are positive-labelled fragments like 135.0.0.22.118. The source labels these inconsistently, which caps what any model can achieve.
- With 46 test rows and 12 negatives, the results are unstable and shouldn't be used to claim one method is better.

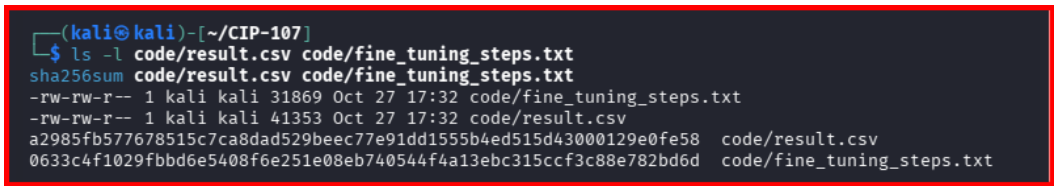
Phase D — Investigate Historical Fine-Tuning Evidence

Step 1: Check that the log files are present and unchanged

cd ~/CIP-107

ls -l code/result.csv code/fine_tuning_steps.txt

sha256sum code/result.csv code/fine_tuning_steps.txt

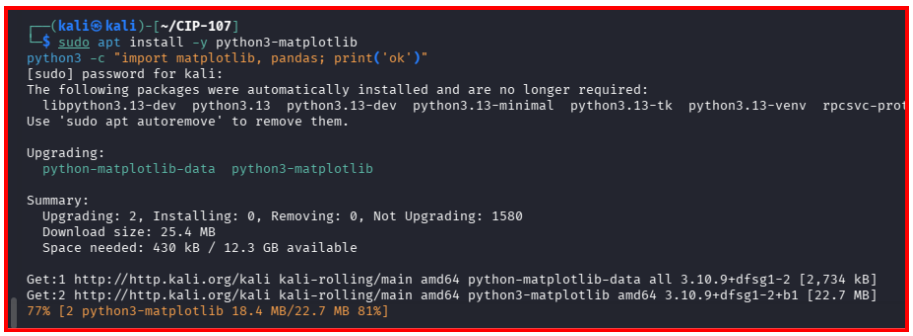


File	SHA-256 (matches)
code/result.csv	a2985fb577678515c7ca8dad529beec77e91dd1555b4ed515d43000129e0fe58
code/fine_tuning_steps.txt	0633c4f1029fbbd6e5408f6e251e08eb740544f4a13ebc315ccf3c88e782bd6d

Step 2: Install the plotting library

sudo apt install -y python3-matplotlib

python3 -c "import matplotlib, pandas; print('ok')"



Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Step 3: Put the script in place (download phase_d_analysis.py)

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ nano phase_d_analysis.py

┌──(kali㉿kali)-[~/CIP-107]
└─$ python3 -c "import ast; ast.parse(open('phase_d_analysis.py').read()); print('syntax ok')"
syntax ok
```

```
 GNU nano 9.0                                    phase_d_analysis.py
"""CIP-107 Phase D — analyse HISTORICAL fine-tuning logs (read-only; nothing is trained or sent to any API).
Usage: python3 phase_d_analysis.py code/result.csv code/fine_tuning_steps.txt [outdir]
"""
import sys, os, json
import numpy as np, pandas as pd, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

res_path, steps_path = sys.argv[1], sys.argv[2]
out = sys.argv[3] if len(sys.argv) > 3 else "phase_d_outputs"
os.makedirs(out, exist_ok=True)
r, s = pd.read_csv(res_path), pd.read_csv(steps_path)

# ── 13. what each file contains, and are they the same run? ──
m = s.merge(r, on="step", suffixes=("_s", "_r"))
summary = {
 "fine_tuning_steps.txt": {"rows": len(s), "columns": list(s.columns), "last_step": int(s.step.max())},
 "result.csv": {"rows": len(r), "columns": list(r.columns), "last_step": int(r.step.max()),
                "validation_rows_present": int(r.validation_loss.notnull().sum()),
                "classification_rows_present": int(r["classification/accuracy"].notnull().sum())},
 "same_run_check": {"matching_steps": len(m),
                    "max_abs_training_loss_difference": round(float((m.training_loss_s - m.training_loss_r).abs().max()), 4),
                    "step1_loss_steps_txt": round(float(s.training_loss.iloc[0]), 4),
                    "step1_loss_result_csv": round(float(r.training_loss.iloc[0]), 4)}}
for name, d in [("fine_tuning_steps.txt", s), ("result.csv", r)]:
    summary[name].update({"mean_loss_first50": round(float(d.training_loss.iloc[:50].mean()), 4),
                          "mean_loss_last50": round(float(d.training_loss.iloc[-50:].mean()), 4),
                          "min_loss": round(float(d.training_loss.min()), 5),
                          "steps_with_train_seq_acc_0": int((d.training_sequence_accuracy == 0).sum()),
                          "mean_train_seq_acc": round(float(d.training_sequence_accuracy.mean()), 4)})
v = r[r.validation_loss.notnull()]
summary["validation"] = {"points": len(v), "min_val_loss": round(float(v.validation_loss.min()), 5),
                         "min_val_loss_step": int(v.step[v.validation_loss.idxmin()]),
```

Step 4: Run it

python3 phase_d_analysis.py code/result.csv code/fine_tuning_steps.txt phase_d_outputs

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ python3 phase_d_analysis.py code/result.csv code/fine_tuning_steps.txt phase_d_outputs
{
 "fine_tuning_steps.txt": {
  "rows": 734,
  "columns": [
   "step",
   "elapsed_tokens",
   "elapsed_examples",
   "training_loss",
   "training_sequence_accuracy",
   "training_token_accuracy"
  ],
  "last_step": 734,
  "mean_loss_first50": 0.0478,
  "mean_loss_last50": 0.0066,
  "min_loss": 0.00134,
  "steps_with_train_seq_acc_0": 42,
  "mean_train_seq_acc": 0.9428
 },
 "result.csv": {
  "rows": 733,
  "columns": [
   "step",
   "elapsed_tokens",
   "elapsed_examples",
   "training_loss",
   "training_sequence_accuracy",
   "training_token_accuracy",
   "validation_loss",
   "validation_sequence_accuracy",
   "validation_token_accuracy",
   "classification/accuracy",
   "classification/precision",
   "classification/recall",
   "classification/auroc",
   "classification/auprc",
   "classification/f1.0"
  ],
  "last_step": 733,
  "validation_rows_present": 92,
  "classification_rows_present": 4,
```

```
 "validation": {
  "points": 92,
  "min_val_loss": 0.00136,
  "min_val_loss_step": 689,
  "final_val_loss": 0.00189,
  "steps_with_val_seq_acc_below_1": [
   1,
   33,
   49,
   65,
   105,
   113,
   121,
   329,
   345,
   577
  ]
 },
 "final_non_null_classification_metrics": {
  "accuracy": 0.978261,
  "precision": 0.971429,
  "recall": 1.0,
  "f1.0": 0.985507,
  "auroc": 0.997549,
  "auprc": 0.999147,
  "step": 733
 },
 "missing_values": {
  "validation_loss": 641,
  "validation_sequence_accuracy": 641,
  "validation_token_accuracy": 641,
  "classification/accuracy": 729,
  "classification/precision": 729,
  "classification/recall": 729,
  "classification/auroc": 729,
  "classification/auprc": 729,
  "classification/f1.0": 729
 }
}

All non-null classification rows:
  step  classification/accuracy  classification/precision  classification/recall  classification/f1.0  classification/auroc  classification/auprc
 185                  0.934783                  0.969697               0.941176             0.955224              0.995098              0.998268
 370                  0.978261                  0.971429               1.000000             0.985507              1.000000              1.000000
 551                  0.978261                  0.971429               1.000000             0.985507              0.997549              0.999147
 733                  0.978261                  0.971429               1.000000             0.985507              0.997549              0.999147

Saved to phase_d_outputs/: phase_d_plots.png, classification_rows.csv, phase_d_summary.json
```

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Run `xdg-open phase_d_outputs/phase_d_plots.png`



Step 5: Hash the outputs:

```
sha256sum phase_d_analysis.py phase_d_outputs/* > PhaseD_manifest.txt
cat PhaseD_manifest.txt
```

```
┌──(kali㉿kali)-[~/CIP-107]
└─$ sha256sum phase_d_analysis.py phase_d_outputs/* > PhaseD_manifest.txt
cat PhaseD_manifest.txt
39e23e2b595d5e607af86bbb7163f0c036f324ac866769a9e6c7a8918d533c6a  phase_d_analysis.py
e2bc2882530144dbb661eaa65cad8ab509e9895105b41a7d22b850dfe3af941b  phase_d_outputs/classification_rows.csv
8b40d52b8f28ac2f7b884f03a364726574be9f7dc5c2d72546c3b69b5947bc86  phase_d_outputs/phase_d_plots.png
9503db71f038b7bd37b10cc7ce594efe31ea80602464dfb534b1f994e3e5e452  phase_d_outputs/phase_d_summary.json
```

Phase D Evidence Integrity Table (Kali run)

Input logs were hashed before analysis, and the script was run read-only on them (no training, no API calls). Outputs were hashed with sha256sum and recorded in PhaseD_manifest.txt. Every figure printed on the analyst machine matched the figures reported above.

File	Description	SHA-256
code/result.csv	Input: historical log, 733 steps, 15 columns	a2985fb577678515c7ca8dad529beec77e91dd1555b4ed515d43000129e0fe58
code/fine_tuning_steps.txt	Input: historical log, 734 steps, 6 columns	0633c4f1029fbbd6e5408f6e251e08eb740544f4a13ebc315ccf3c88e782bd6d
phase_d_analysis.py	Analysis script	39e23e2b595d5e607af86bbb7163f0c036f324ac866769a9e6c7a8918d533c6a

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

phase_d_outputs/classification_rows.csv	The 4 non-null classification rows	e2bc2882530144dbb661eaa65cad8ab509e9895105b41a7d22b850dfe3af941b
phase_d_outputs/phase_d_plots.png	Four-panel plot	8b40d52b8f28ac2f7b884f03a364726574be9f7dc5c2d72546c3b69b5947bc86
phase_d_outputs/phase_d_summary.json	Summary statistics and missing-value counts	9503db71f038b7bd37b10cc7ce594efe31ea80602464dfb534b1f994e3e5e452

Forensic integrity statement: the historical logs were read in place and not modified. Hashing shows the files and outputs have not changed since they were produced; it does not show which fine-tuning job produced either log, which remains unconfirmed.

Phase E — Validate, Investigate and Interpret

E1. Actual-versus-Predicted Classification Table

The positive class is defined as **positive (1)**. Using the TF-IDF + Logistic Regression classifier on the 46-record holdout set, the confusion matrix was reported as **[[11,1],[0,34]]**, corresponding to 11 TN, 1 FP, 0 FN and 34 TP.

Actual Class	Predicted Class	Outcome	Count
Negative (0)	Negative (0)	True Negative (TN)	11
Negative (0)	Positive (1)	False Positive (FP)	1
Positive (1)	Negative (0)	False Negative (FN)	0
Positive (1)	Positive (1)	True Positive (TP)	34
Total			**46**

Confusion Matrix

	Predicted Negative	Predicted Positive
Actual Negative	11	1
Actual Positive	0	34

The classifier correctly classified 45 of 46 holdout records, with a single false positive and no false negatives.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

E2. Performance Metrics

Using the confusion matrix:

- TN = 11
- FP = 1
- FN = 0
- TP = 34

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$
$$= \frac{34 + 11}{46} = \frac{45}{46} = 0.978$$

Accuracy = 97.8%

Precision

$$Precision = \frac{TP}{TP + FP}$$
$$= \frac{34}{35} = 0.971$$

Precision = 97.1%

Recall

$$Recall = \frac{TP}{TP + FN}$$
$$= \frac{34}{34} = 1.000$$

Recall = 100.0%

F1 Score

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$
$$= \frac{2 \times 0.971 \times 1.000}{0.971 + 1.000} = 0.986$$

F1 = 98.6%

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

These values match the classifier evaluation reported in Phase C.

Majority-Class Baseline

The holdout contains:

- 34 positive records
- 12 negative records

A classifier that always predicts **positive** would produce the following confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	0	12
Actual Positive	0	34

Resulting metrics:

Metric	Value
Accuracy	73.9%
Precision	73.9%
Recall	100.0%
F1	85.0%

The TF-IDF classifier outperforms this baseline substantially, although caution is required because 73.9% of the test records are exact or near-duplicates of training examples.

AUROC and AUPRC

Probability scores were available for the classifier. Historical classification logs report the following values at the final evaluation point:

- AUROC = 0.997549
- AUPRC = 0.999147

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

However, these values originate from a historical fine-tuning evaluation rather than the reproducible offline classifier and should therefore be reported separately from the Phase C results.

E3. Analyst Review of Holdout Records

Record 1 – 608.80.24.1

Source label: Negative
Classifier prediction: Negative

The candidate originates from the string 3608.80.24.1.8. The first octet exceeds IPv4 limits and is therefore invalid. The negative classification is technically correct and consistent with Phase B findings concerning contaminated dotted-number sequences.

Outcome: True Negative

Record 2 – 257.0.1.38

Source label: Negative
Classifier prediction: Negative

The value contains octet 257, which exceeds the maximum permitted value of 255. Phase B specifically identified similar cases as invalid IPv4 candidates.

Outcome: True Negative

Record 3 – 0.0.0.0

Source label: Negative
Classifier prediction: Negative

Although syntactically valid, 0.0.0.0 is an unspecified placeholder address rather than a meaningful endpoint. Phase B documented such entries as ambiguous placeholders and the rule baseline explicitly rejects them.

Outcome: True Negative

Record 4 – 2.20.31.4

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Source label: Positive
Classifier prediction: Positive

The address is a clean standalone IPv4 candidate with no signs of contamination or truncation. Both classifier and rule baseline classify it as positive.

Outcome: True Positive

Record 5 – 20.10.0.8

Source label: Positive
Classifier prediction: Positive

This record appears as a standalone IPv4 address and contains no obvious structural issues. The classification is reasonable and aligns with the source label.

Outcome: True Positive

Record 6 – 255.255.255.255

Source label: Positive
Classifier prediction: Positive

This is the IPv4 broadcast address. Although not a routable host address, it is a syntactically valid IPv4 value. The classifier follows the source label correctly.

Outcome: True Positive

Record 7 – 10.5.0.1 within a Belkin URL

Source label: Positive
Classifier prediction: Positive

The candidate appears within:

http://.../10.5.0.1.6.137/...

This appears more consistent with a software version string than a standalone IP address. Phase B highlighted version-like contexts as a source of ambiguity. The classifier follows the label, but an analyst could reasonably challenge that label.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Outcome: Borderline / Ambiguous Positive

Record 8 – 135.0.0.22 extracted from 135.0.0.22.118

Source label: Positive
Classifier prediction: Positive

The candidate is only part of a longer dotted sequence. The rule baseline rejects such cases using rule R5, whereas the classifier predicts positive because that pattern appears in the training labels. This represents a likely labelling inconsistency rather than a clear classifier success.

Outcome: Borderline / Ambiguous Positive

Review Summary

The investigation reveals that most classifier decisions reflect the source labels accurately. However, several positive-labelled records appear to be version numbers, OID fragments or portions of longer dotted sequences rather than genuine standalone IP addresses. Phase B documented similar inconsistencies throughout the dataset, suggesting that source-labelling quality places an upper bound on achievable classifier performance.

Furthermore, Phase C showed that 34 of 46 holdout records are exact or near-duplicates of training records, meaning that much of the reported accuracy may reflect memorisation rather than genuine generalisation.

E4. Why High Accuracy Can Still Be Operationally Poor

Accuracy can be misleading when classes are imbalanced.

Consider a forensic triage dataset containing:

- 990 benign artefacts
- 10 malicious artefacts

A classifier that predicts every record as benign achieves:

$$\frac{990}{1000} = 99\%$$

accuracy.

Despite achieving 99% accuracy, the system fails to identify any of the malicious artefacts. Operationally, such a model is ineffective because it completely misses the events analysts actually care about.

False-Negative Consequences in Forensic Triage

A false negative may result in:

- Malicious infrastructure being overlooked.
- Indicators of compromise remaining undetected.
- Delayed incident response.
- Missed investigative leads.
- Additional victim systems remaining exposed.

In digital forensics, false negatives are often more serious than false positives because a missed artefact may never receive further review.

Appropriate Analyst Escalation

When a record appears technically suspicious but receives a negative classification:

1. Validate the IP candidate manually.
2. Review the surrounding text and extraction context.
3. Compare classifier and rule-based outputs.
4. Check for contamination, version strings and truncation effects.
5. Escalate uncertain records for analyst review rather than automatically discarding them.
6. Document the rationale for any override decision.

Accordingly, forensic workflows should prioritise analyst review of uncertain cases and maintain high recall wherever possible, even if this results in additional false positives.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Phase F — Capstone Intelligence Decision

Forensic Intelligence Memorandum

Remit

The objective of this investigation was to evaluate methods for identifying IPv4 candidates from textual artefacts and to assess whether machine learning or historical fine-tuning evidence provides a reliable basis for forensic triage. The investigation considered evidence preservation, candidate extraction, offline classification, historical training records and analyst review. The purpose was not to determine whether any recovered IP address was malicious, but rather whether the extracted candidate was likely to represent an IPv4 address according to the dataset definitions and supporting evidence.

Evidence Manifest

Evidence Item	Purpose
IPs_500_SQLite_dataset.csv	Labelled source dataset used for extraction and analysis
training400_dataset.jsonl / dataset_train.jsonl	Training data for offline classifier
testing100_dataset.csv / dataset_test.jsonl	Holdout evaluation data
candidate_review.csv	Phase B candidate extraction results
phase_c_classifier.py	Offline ML classifier
test_predictions.csv	Per-record predictions and leak flags
result.csv	Historical fine-tuning metrics
fine_tuning_steps.txt	Historical training log

All evidence and outputs were protected through SHA-256 hashing and documented in the corresponding phase manifests.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Methodology

Phase B: Extraction and Validation

1. Regex extraction identified candidate dotted-quad strings.
2. Python IPv4 validation assessed syntactic correctness.
3. Contextual analysis identified:
    - Version strings
    - OID fragments
    - URL components
    - Placeholder addresses
    - Contaminated dotted sequences

The purpose was to distinguish between simple pattern matching and genuine IPv4 candidates.

Phase C: Offline Classification

A completely offline classifier was trained using:

- TF-IDF character n-grams (2-5)
- Logistic Regression
- Balanced class weighting
- Fixed random seed (42)

Performance was compared against a transparent rule-based baseline. Leakage analysis was also performed to assess train-test overlap.

Phase D: Historical Fine-Tuning Investigation

Historical OpenAI fine-tuning logs were examined as evidence artefacts. No fine-tuning was re-run and no cloud-based model was accessed. The analysis focused on understanding the provenance and limitations of the historical results.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Results Summary

Method	Accuracy	Precision	Recall	F1
TF-IDF + Logistic Regression	97.8%	97.1%	100.0%	98.6%
Rule Baseline	78.3%	100.0%	70.6%	82.8%
Always Positive Baseline	73.9%	73.9%	100.0%	85.0%

The classifier correctly classified 45 of 46 holdout records with one false positive and no false negatives. However, 73.9% of holdout records were exact or near-duplicates of training records, creating a substantial memorisation risk.

Eight-Record Analyst Review

Candidate	Source Label	Model Result	Assessment
608.80.24.1	Negative	Negative	Invalid octet structure
257.0.1.38	Negative	Negative	Invalid IPv4 (>255 octet)
0.0.0.0	Negative	Negative	Placeholder address
2.20.31.4	Positive	Positive	Standalone valid IPv4
20.10.0.8	Positive	Positive	Standalone valid IPv4
255.255.255.255	Positive	Positive	Valid broadcast address
10.5.0.1 (Belkin URL)	Positive	Positive	Likely version/URL fragment
135.0.0.22 (from 135.0.0.22.118)	Positive	Positive	Likely partial dotted sequence

The review showed that several source-labelled positives appear to be version strings or fragments of longer identifiers rather than clear standalone IPv4 addresses. This indicates limitations in source-labelling consistency.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Limitations

1. High train-test overlap (73.9%) limits confidence in generalisation. [CIP-107_Ph...er_v3.docx | Word]
2. Holdout size is small (46 records, 12 negatives). [CIP-107_Ph...er_v3.docx | Word]
3. Some source labels appear internally inconsistent. [CIP-107_Ph...on_v2.docx | Word], [CIP-107_Ph...er_v3.docx | Word]
4. Historical fine-tuning logs lacked model artefacts, configuration details and reproducible workflows. [CIP-107_Ph...ce_v2.docx | Word]
5. No threat-intelligence enrichment was performed.
6. No recovered address can be classified as malicious solely from these datasets.

Conclusion

The evidence demonstrates that syntactic IPv4 detection can be performed reliably using a combination of regex extraction, IPv4 validation and contextual analysis. The offline classifier achieved very strong measured performance but the high degree of holdout duplication means the reported accuracy most likely overstates real-world effectiveness. Historical fine-tuning logs show successful previous training activity but do not provide independently reproducible evidence of superior performance.

No recovered IP address should be interpreted as malicious without independent corroborating evidence such as network logs, incident artefacts, threat-intelligence reporting or investigative context.

Recommended Next Investigative Actions

1. Create a genuinely independent test set with no prompt overlap.
2. Review and correct inconsistent source labels.
3. Expand the negative class with additional examples.
4. Validate extracted IPs against external intelligence sources.
5. Test analyst agreement on ambiguous cases.
6. Maintain manual review for uncertain records.
7. Document confidence levels rather than binary decisions.
8. Treat all extracted candidates as investigative leads only.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Comparative Assessment of Available Approaches

Approach	Strengths	Limitations
Regex + IPv4 Validation	Transparent, explainable, reproducible, immediate detection of malformed addresses	Cannot distinguish IPs from version strings or contextual artefacts
Offline ML Classifier	Higher measured performance, learns contextual patterns automatically	Leakage risk, limited explainability, requires high-quality training data
Historical Fine-Tuning Results	Demonstrates earlier successful model training and strong recorded metrics	Not reproducible, not independently verifiable, dependent on retired infrastructure

Deployment Recommendation

The preferred operational workflow is:

Regex Extraction → IPv4 Validation → Offline ML Scoring → Human Analyst Review

The offline ML model should be deployed **only as analyst assistance**, not as an autonomous decision-maker.

This recommendation is justified because:

- Dataset labelling inconsistencies remain unresolved.
- Holdout leakage reduces confidence in reported accuracy.
- Several positive-labelled examples remain ambiguous.
- Forensic decisions require evidential interpretation rather than pattern recognition alone.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Analyst Review Decision Matrix

The objective of this matrix is to convert extraction and classification results into operational forensic decisions. Classification confidence alone is insufficient because many candidate strings resemble IPv4 addresses while actually representing version numbers, OIDs, URLs or truncated identifiers. Analyst review remains mandatory before any intelligence judgement.

Candidate Example	Source Label	Classifier Result	Technical Validity	Context Assessment	Analyst Confidence	Recommended Action
2.20.31.4	Positive	Positive	Valid IPv4	Standalone address	High	Retain as candidate IP artefact
20.10.0.8	Positive	Positive	Valid IPv4	Standalone address	High	Retain as candidate IP artefact
255.255.255.255	Positive	Positive	Valid IPv4	Broadcast address	Medium	Retain but annotate special-purpose address
0.0.0.0	Negative	Negative	Valid IPv4 syntax	Placeholder address	High	Exclude from investigative leads
257.0.1.38	Negative	Negative	Invalid IPv4	Octet exceeds 255	High	Reject as malformed artefact
608.80.24.1	Negative	Negative	Invalid IPv4	Contaminated dotted sequence	High	Reject as malformed artefact
10.5.0.1 (Belkin URL)	Positive	Positive	Valid IPv4 syntax	Embedded within version-like URL path	Low	Escalate for analyst review
135.0.0.22 (from 135.0.0.22.118)	Positive	Positive	Valid IPv4 syntax	Appears to be part of longer dotted sequence	Low	Escalate for analyst review

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Analyst Decision Categories

Category	Criteria	Action
Accept	Valid IPv4, standalone context, classifier and analyst agree	Retain as investigative artefact
Reject	Invalid IPv4, malformed extraction, placeholder value or obvious non-IP artefact	Remove from candidate set
Escalate	Valid syntax but ambiguous context, label disagreement, version-string indicators, URL embedding or contamination	Manual review required
Corroborate	Candidate appears genuine and potentially relevant to investigation	Seek supporting evidence such as logs, DNS, firewall records, EDR telemetry or threat intelligence

---

Intelligence Decision Framework

To support operational deployment, each recovered candidate should progress through the following decision path:

Regex Extraction

↓

IPv4 Validation

↓

Offline ML Classification

↓

Analyst Context Review

↓

┌──────────────┬──────────────┬──────────────┐

│ Accept │ Reject │ Escalate │

└──────────────┴──────────────┴──────────────┘

↓

Independent Corroboration

↓

Intelligence Assessment

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Intelligence Thresholds

Finding	Intelligence Judgement
Valid IPv4 only	Candidate network artefact
Valid IPv4 + suspicious context	Investigative lead
Valid IPv4 + corroborating forensic evidence	Potential indicator requiring investigation
Valid IPv4 + threat-intelligence confirmation	Intelligence-relevant artefact
Valid IPv4 alone	Not sufficient to classify as malicious

Final Capstone Decision

Recommended deployment status: Analyst Assistance Tool

Approved uses

- Candidate extraction
- Candidate prioritisation
- Triage support
- Analyst workload reduction
- Consistency checking

Not approved uses

- Autonomous threat attribution
- Automatic maliciousness determination
- Automatic evidence exclusion
- Sole basis for investigative decisions

This decision is justified because the dataset contains documented label inconsistencies, the holdout set contains substantial train-test overlap (73.9%), and several positive-labelled examples remain contextually ambiguous. The correct operational role is therefore decision support for a human analyst, not automated forensic judgement.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Executive Briefing (Five-Slide Equivalent)

Slide 1 – Objective

Goal: Evaluate methods for identifying IPv4 candidates in forensic artefacts.

Methods Assessed

- Regex extraction
- IPv4 validation
- Offline machine learning
- Historical fine-tuning evidence

Key Finding

- Candidate detection is feasible, but context matters.

Slide 2 – Evidence and Methodology

Evidence

- Labelled SQLite dataset
- Offline training and test sets
- Historical fine-tuning logs

Approach

1. Extract candidates
2. Validate IPv4 syntax
3. Train offline classifier
4. Compare against rule baseline
5. Conduct analyst review

Slide 3 – Results

Model	Accuracy	Precision	Recall
TF-IDF + Logistic Regression	97.8%	97.1%	100.0%

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Rule Baseline	78.3%	100.0%	70.6%
Always Positive	73.9%	73.9%	100.0%

Important Caveat

73.9% of holdout records were exact or near-duplicates of training examples. Reported accuracy should therefore be interpreted cautiously.

Slide 4 – Key Risks

- Small evaluation dataset.
- Label inconsistencies.
- Significant train-test overlap.
- Several positives resemble version strings rather than standalone IPs.
- Historical fine-tuning results cannot be independently reproduced.

Slide 5 – Intelligence Assessment

Judgement

- Regex and IPv4 validation provide robust first-stage filtering.
- Offline ML provides useful prioritisation capability.
- Historical fine-tuning evidence should be treated as background information only.

Recommendation

Deploy as:

Analyst Assistance Tool

Not:

Autonomous Intelligence Classification System

Intelligence Caveat

No recovered address should be designated malicious without corroborating technical or investigative evidence.

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Appendix — Supporting Reference

This appendix consolidates references already documented in the report. It does not add new evidence, independently verify hashes, or establish the provenance of the historical fine-tuning logs.

A. Evidence and Output Index

- **Phase A — Preserved evidence:** IPs_500_SQLite_dataset.csv; IPs_6499_SQLite_dataset.xlsx; ReadMe.txt; testing100_results_dataset.csv; testing100_dataset.csv; training400_dataset.jsonl. Refer to the Evidence Integrity Table (Phase A) for their recorded SHA-256 hashes.
- **Phase B — Extraction:** phase_b_extract.py and demo.py; candidate_review.csv and phase_b_demo_samples.csv. Verification records: SHA256_manifest.txt and PhaseB_manifest.txt.
- **Phase C — Offline evaluation:** phase_c_classifier.py; dataset_train.jsonl and dataset_test.jsonl; classification_reports.txt, environment.txt, metrics.json, and test_predictions.csv. Integrity record: PhaseC_manifest.txt. The predictions file includes leak flags and rule reasons.
- **Phase D — Historical evidence:** result.csv and fine_tuning_steps.txt; phase_d_analysis.py; classification_rows.csv, phase_d_plots.png, and phase_d_summary.json. Integrity record: PhaseD_manifest.txt. Input logs were analysed read-only; no training or API calls were performed.

B. Evaluation Reference and Caveats

- **Positive class:** positive (1). The reported offline-classifier confusion matrix is [[11,1],[0,34]]: 11 true negatives, 1 false positive, 0 false negatives, and 34 true positives.
- **Recorded metrics:** Accuracy 97.8%; precision 97.1%; recall 100.0%; F1 98.6%. These are the Phase C holdout results, not the historical fine-tuning results.
- **Evaluation limits:** 33 of 46 holdout rows were exact duplicates of training prompts; 34 of 46 were exact or near-identical at cosine similarity ≥ 0.90. The clean subset contained 12 records, and the classifier misclassified its only negative example. The small sample and overlap limit confidence in generalisation.
- **Historical results:** Final recorded AUROC 0.997549 and AUPRC 0.999147 belong to the historical fine-tuning evaluation. The producing job remains unconfirmed, and these metrics must not be presented as reproducible offline-classifier results.

Academic Integrity and Authorization Statement

I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

C. Analyst Decision Reference

- **Accept:** Retain a valid, standalone IPv4 candidate when the classifier and analyst agree.
- **Reject:** Remove malformed, placeholder, or obvious non-IP candidates from the candidate set following analyst review; do not automatically exclude original evidence.
- **Escalate:** Manually review valid-looking candidates with ambiguous context, label disagreement, version-string indicators, URL embedding, or contamination.
- **Corroborate:** Seek supporting logs, DNS records, firewall records, EDR telemetry, or threat intelligence for potentially relevant candidates. IPv4 validity alone does not establish maliciousness.

Operational status: Analyst Assistance Tool only—not an autonomous intelligence classification system.

D. Terminology

- **,IPv4 candidate:** A dotted-quad string extracted from textual artefacts for syntactic validation and contextual review; validity alone does not establish maliciousness. *(See: Phase B — IP Candidate Extraction and Labelling; Phase F — Remit.)*
- **Positive class (1):** The dataset's positive classification for a candidate likely to represent an IPv4 address, not a designation of maliciousness. *(See: E1. Actual-versus-Predicted Classification Table; Phase F — Remit.)*
- **True Negative (TN) / True Positive (TP):** Records whose predicted negative or positive class matches the actual class. *(See: E1. Actual-versus-Predicted Classification Table.)*
- **False Positive (FP) / False Negative (FN):** An actual negative predicted as positive, or an actual positive predicted as negative, respectively. *(See: E1. Actual-versus-Predicted Classification Table; E4. Why High Accuracy Can Still Be Operationally Poor.)*
- **Train-test overlap (leakage):** Exact or near-identical prompts shared between training and holdout data, limiting confidence in generalisation. *(See: Phase C summary table — Fixed holdout and leakage check; E3. Analyst Review of Holdout Records — Review Summary.)*
- **Rule-based baseline:** The transparent comparison method that applies explicit structure, IPv4-validity, placeholder, and context rules to candidates. *(See: Phase C summary table — Rule-based baseline.)*

- **Placeholder address:** An entry such as `0.0.0.0`, described in the report as syntactically valid but unspecified rather than a meaningful endpoint. *(See: E3. Analyst Review of Holdout Records — Record 3 – 0.0.0.0.)*
- **Analyst Assistance Tool:** Decision support for candidate extraction, prioritisation, and triage under human review—not autonomous threat attribution or maliciousness determination. *(See: Phase F — Final Capstone Decision; Deployment Recommendation.)*

Investigator Sign-Off

Investigator Name: Favour Madu

Investigator Signature:

Date:

Academic Integrity and Authorization Statement
I, Favour Madu, confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.